S2 E12

BK

Season 2: Retrieval and RAG

Field note

The embedding swap.

S1 E3 said: treat the embedding model as a dependency, and test it on your own questions before you change it. This field note does exactly that, with a bigger, pricier model.

Brendan Kelly

Applied AI

S2 E12

Field note

This episode is a field note

No new idea this time. A field note takes a lesson already taught and puts it to a full test.

LESSON FROM S1 E3

An embedding is a point in a space of meaning

Search turns each passage and each question into numbers, and returns the passages that sit closest.

ITS DECISION RULE

Pin it, test it, re-embed

"Treat the embedding model as a versioned dependency: pin it, test it on your own queries, and re-embed when it changes."

The open question

S1 E3 gave the rule, but never ran it. What actually happens to a whole RAG system when you swap the embedding model?

HOW TO READ IT

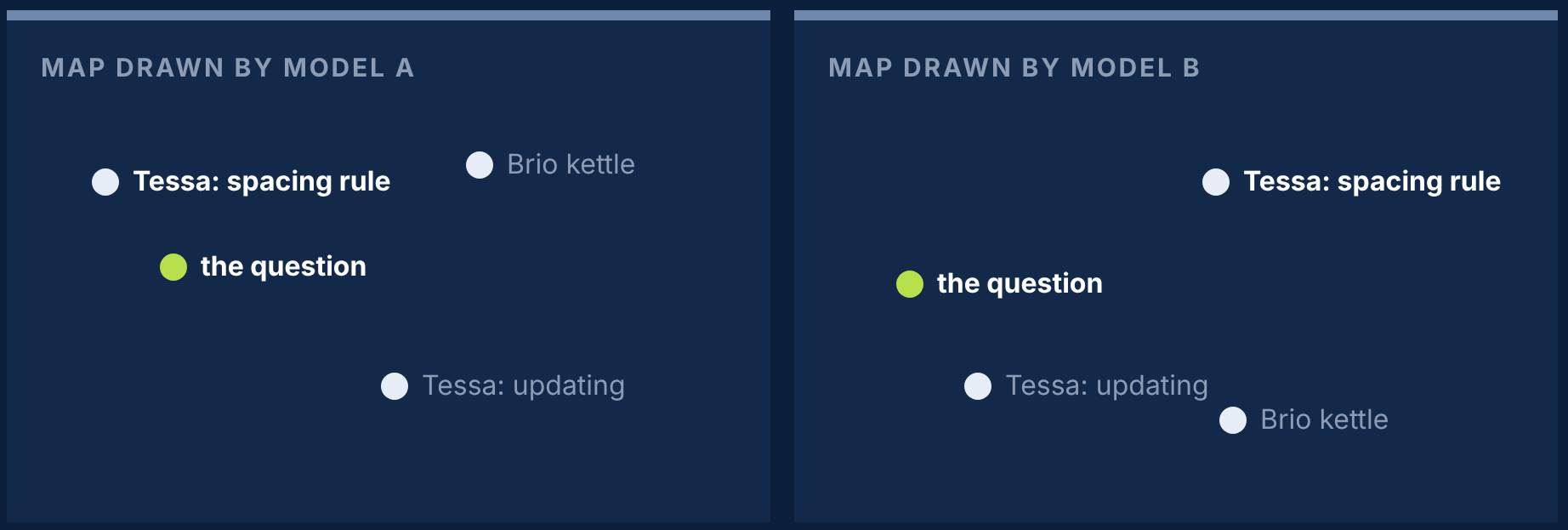
Question → Setup → Result → Limits → Decision rule

S2 E12

Refresher

Refresher: what an embedding model does

From S1 E3. An embedding model turns text into a list of numbers: a point on a map of meaning. Texts that mean similar things land close together.



Search

Returns the passages that sit closest to the question on the map.

Each model draws its own map

Change the model, and every distance changes. That is why a swap means re-embedding the whole library, and why it can help or hurt.

Illustration only: real maps have over a thousand dimensions.

Why it matters

Where stage 3 sits, and why swapping it is tempting

1	2	**3**	4	5	6	7	8	9	10	11
Load **OFFLINE**	Chunk **OFFLINE**	**Embed BOTH**	Rewrite **LIVE**	Search **LIVE**	Filter **LIVE**	Rerank **LIVE**	Prompt **LIVE**	Answer **LIVE**	Cite **LIVE**	Score **LIVE**

offline: once, when documents are added **live**: on every question **both**: the library offline, each question live

3. Embed
turns every passage, once, and every question, live, into numbers

→

5. Search
finds the passages whose numbers sit closest

→

7. Rerank
reads the top 20 and picks the best 5

The temptation	Providers sell larger embedding models. Bigger, newer and pricier sounds like better search, so swapping feels like a free upgrade.
The catch	Every passage was embedded by the old model. Change models, and the whole library must be embedded again, with the questions.

This field note

Does a bigger embedding model find more, and what does the switch really cost?

The test: run the system once with each model

All 520 test questions go through the full system twice. The only difference is which model draws the map at stage 3.

1 **text-embedding-3-small** the reference
The model the whole season was built on. $0.02 per million tokens.

2 **text-embedding-3-large**
OpenAI's larger model, 6.5 times the price: $0.13 per million tokens. Every passage and every question embedded again with it.

Kept the same	Search, the reranker, the prompt and the answer model. They were set up with the small model and not adjusted for the large one.

When would the large model replace the small one?

We wrote the answer down before the run, so the result could not bend it. Two conditions: it must find the right passage more often, by more than luck could explain, and it must not give fewer right answers.

What was expected, written before the run

Three predictions, each with a pass mark set in advance.

Bigger finds more	The large model puts the right passage in the top 5 more often.
Totals hide a reshuffle	An older experiment found that totals barely moved, while many single questions changed hands. Expect the same here.
Search drives answers	If search gets better or worse, right answers move the same way.

How to read the results

"Found" means the right passage reached the top 5, the passages the model is given. Every comparison is question by question.

Finding

The bigger model found less, and answered worse

The full system, all 520 test questions. Small against large.

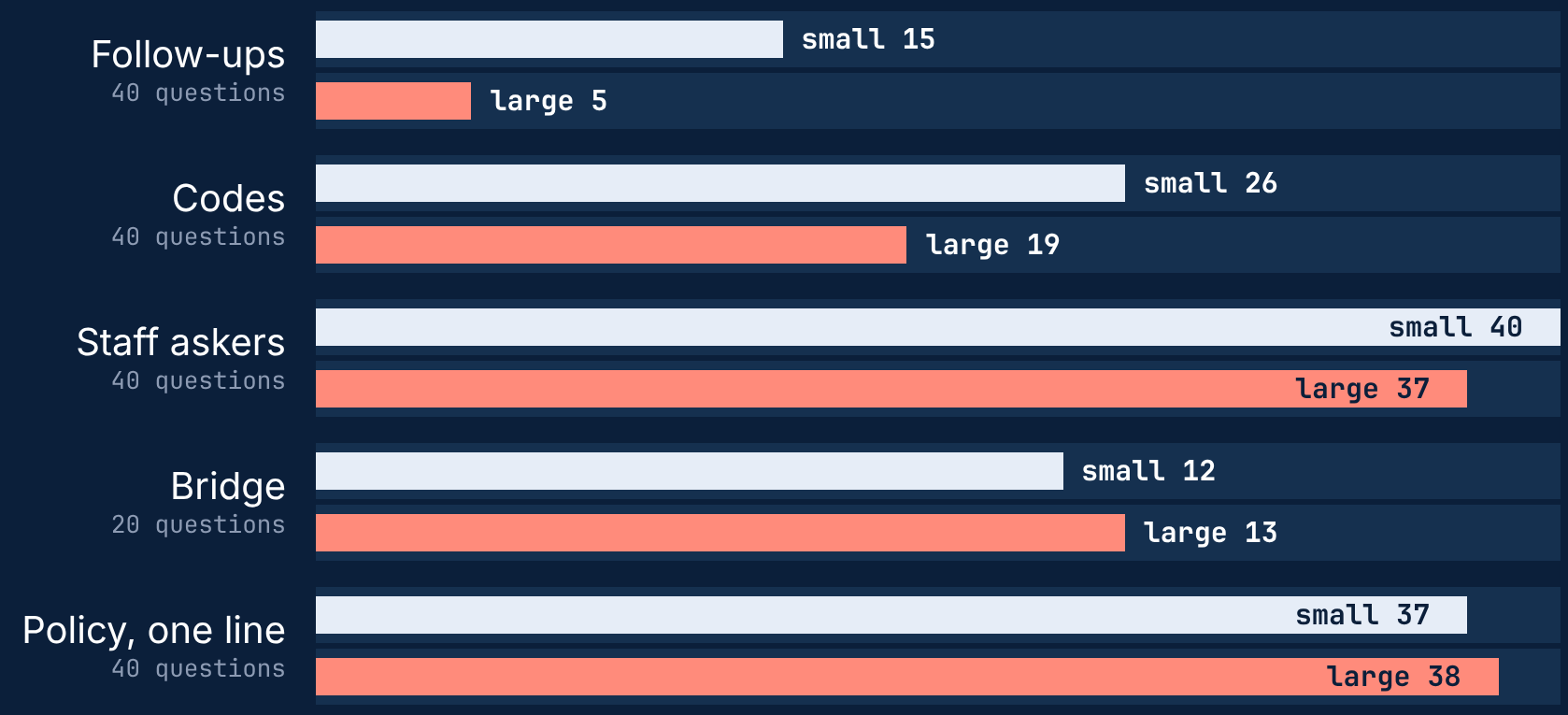
	FOUND	RIGHT ANSWERS	SAID NOT FOUND	LEAKS
3-small the reference	**344** of 400	**354** of 400	**107** of 120	**0** of 480
3-large	**326** -18	**337** -17	**106** -1	**0** of 480

Found, one by one	Of the 400: 29 found only by small, 11 only by large. A split that uneven happens by chance about 1 time in 160.
Right, one by one	31 right only with small, 14 only with large. About 1 time in 60 by chance.

Finding

It lost on follow-ups and part codes

Found in the top 5, by kind of question. Seven other kinds were at or near the top under both models.



So The loss is not spread evenly. It sits on the two kinds of question search already found hardest: short follow-ups (E6) and exact codes (E5).

Worked example

A part code the bigger model never found

A CODE QUESTION FROM THE TEST

What is part KX-9283 for? Answer: the Wi-Fi module for the Ashby tumble dryer

Small	Search ranks the right article 5th; the reranker moves it 1st. "KX-9283 is the Wi-Fi module for the Ashby tumble dryer, which connects the dryer to the app."	right
Large	The article is not among the 20 passages the reranker reads. Its top picks are other parts articles. "I could not find what part KX-9283 is for in the passages."	wrong

Why codes suffer

An embedding captures meaning. "KX-9283" has little meaning, so to a model that leans harder on meaning, every parts article looks alike. E5 showed keyword search handles exact codes better.

Worked example

A follow-up that slipped out of the top 5

A FOLLOW-UP FROM THE TEST

"And the Tessa dishwasher?" after "How much room should I leave around the Brio kettle?"

Small	The safety passage with the spacing rule ranks 1st. "Leave at least 4 centimetres of space around the Tessa dishwasher."	1st
Large	The same passage ranks 12th, below general Tessa articles. "The passages don’t specify how much room to leave around the Tessa dishwasher."	12th

Why follow-ups suffer

Search only sees "And the Tessa dishwasher?". The large model matched the topic, Tessa, not the missing point, spacing. Rewriting the question first (E6) is the real fix.

Finding

Two of three predictions were wrong

The predictions from slide 6, against the result.

Bigger finds more	It found 326 of 400, against 344.	`failed`
Totals hide a reshuffle	40 questions changed hands for a net loss of 18: mostly one way, not a reshuffle.	`failed`
Search drives answers	Search fell by 18 and right answers by 17.	`held`

The lesson

Writing predictions down first is what makes a surprise visible. Without them, "bigger is better" would have been assumed, not tested.

What switching costs, line by line

The large model's price is 6.5 times higher. But price per token is only one line, and the smallest.

	3-SMALL	3-LARGE	PAID
Re-embed the library Halvard, 288,666 tokens	**$0.006**	**$0.04**	once
The same, 1,000x larger about 289 million tokens	**$5.77**	**$37.53**	once
Embed questions per million questions	**~$0.40**	**~$2.40**	ongoing
Rerank and answer per million questions	**~$1,100**	**~$1,100**	ongoing
Retest the system this field note	**$1.71** plus people's time		each swap

So the pattern is

Embedding is a rounding error next to the language models that read and answer: under 0.25% of the running cost, with either model. Price alone should never decide this.

Per-question figures from this run's token counts at gpt-6-luna list prices; "~" marks estimates.

The cost

The cost that decides it: more wrong answers

Switching to the large model saved nothing and cost quality. That is the number to put in front of a decision.

IN THIS TEST

17 fewer right

of 400 answerable questions: about 4 in every 100.

AT 100,000 QUESTIONS A MONTH

~4,250

more wrong answers a month, if the same rate held.

What a wrong answer costs	A support call, a refund, a lost customer, or worse in a regulated setting. Price that, and it dwarfs the embedding bill.
What the test cost	$1.71 here. The cheapest line on the previous slide is the one that caught the problem.

The decision for leaders

Judge a model change by its effect on right answers, priced in your business terms. Pay for the test before you pay for the switch.

The small model stays

The two conditions from slide 5, written before the run. The large model failed both.

Finds more, beyond luck	326 against 344: it found fewer.	`failed`
No fewer right answers	337 against 354: 17 fewer.	`failed`

Not a verdict on the large model

It may do better on other libraries, or with search retuned for it. The point is narrower: on our questions, with our system, it lost. Only a test on your own questions can tell you which way it goes.

What this does not prove

- **Two models from one provider.** Another provider’s model might do better or worse.
- **No retuning.** Search depth, shortlist and prompt were tuned with the small model.
- **One library.** The loss on follow-ups and codes may be particular to Halvard’s documents.
- **The judge** is gemini-3.6-flash, checked against E3’s sample.

For engineers

Run it yourself

Both full runs, and the cost of re-embedding.

```
# rebuild every result and chart: no key, no cost
$ uv run lab rag analyse field-notes/s2-e12-embedding-swap/config.yaml

# the full run (keys needed)
$ uv run lab rag run field-notes/s2-e12-embedding-swap/config.yaml
```

The change	`embed: {model: text-embedding-3-large}`, one line in config.yaml.
Cost	$1.71 for the field note: $0.53 of OpenAI calls, $1.18 of judging.

For leaders

Treat an upgrade as a change to test

Here, a model 6.5 times the price gave 17 fewer right answers. Nobody would have known without a test on our own questions.

Ask your team

Which embedding model do we use, and is its version pinned?

Ask your team

If the provider retired it tomorrow, how would we know the replacement is not worse?

Ask your team

Do we have a test set of our own questions to run before any model change?

Decision rule

Swap embedding models in one step: re-embed everything, switch queries with it, and compare answers question by question.

S1 E3 holds	Bigger is not an upgrade by default. Here it lost, so the old model stays.

Next: S2 E13, a field note

Long context or retrieval? If the whole library fits in the prompt, do you need search at all?

Where this fits

The series map

S1 LLMs	**S2** Retrieval	**S3** Strategy
S4 Adapting	**S5** Agents	**S6** Economics
S7 Data	**S8** Evaluation	**S9** AI teams
S10 Reliability	**S11** Security	**S12** Delivery
S13 Performance	**S14** Resilience	**S15** Governance

☐ Leadership season

Follow linkedin.com/in/brendan-kelly-irl

Episode guide brendankellyai.github.io

Code github.com/BrendanKellyAI/applied-ai-lab